## 第3章 栈和队列

栈和队列本质上也是线性表，是操作受限的线性表。

3.1 栈

3.2 栈的应用

3.3 队列

## 3.1 栈

3.1.1 栈的定义

3.1.2 栈的实现

3.1.3 栈的实现方式的比较

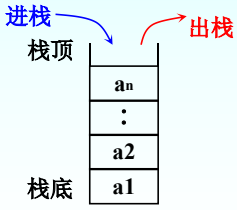
## 3.1.1 栈（Stack）的定义

*栈的定义*

——规定只能在表的一端进行插入和删除的线性表。

允许插入和删除的一端为*栈顶*

不允许插入和删除的一端为*栈底*




➢栈的特点：

后进先出（LIFO）

先进后出（FILO）

例：一个栈的进栈序列是a、b、c、d、e，则栈不可能的输出序列是（　　）。

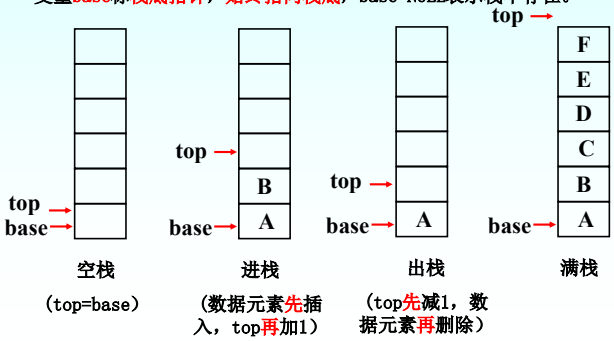
A edcba　　B decba　　C dceab　　D abcde

## 3.1.2 栈的实现——顺序栈和链栈

顺序栈

变量top称栈顶指针，实现进栈和出栈，始终指向栈顶元素的下一个位置；

变量base称栈底指针，始终指向栈底，base=NULL表示栈不存在。




## 顺序栈的进栈（出栈）算法

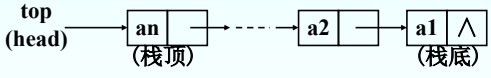
```
Status Push (SqStack&S, SElemType&e)
{
  if (S.top-S.base>=S.stacksize)    //栈已满，追加存储空间
     S.base=（SElemType*）realloc（S.base,（S.stacksize
              +STACKINCREMENT）*sizeof（SElemType））；
  if(！S.base）exit(OVERFLOW);   //存储分配失败
  *S.top++=e;  //插入栈顶元素，top加1
  return OK;
}

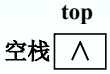
Status Pop (SqStack&S, SElemType&e)
{  if (S.top==S.base) return ERROR; //栈空
   e=*--S.top;  //top减1，删除栈顶元素
   return OK;
}
```

## 链栈

在链表表示的栈中，链表的第一个结点是栈顶结点，最后一个结点是栈底结点。

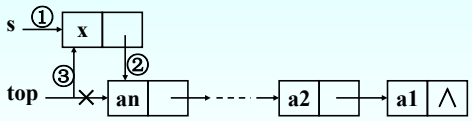
头指针就是栈顶指针top，指向栈顶结点；栈底结点的指针为空。



## 链栈的进栈

*进栈*（在第一个结点前插入值为x的结点）




① s=(LStackNode *)malloc(sizeof(StackNode));

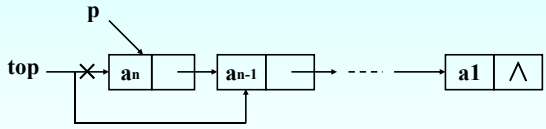
s->data=x;

② s->next=top;　　//top的定义为Node类型

③ top=s;

## 链栈的出栈

*出栈*（删除栈顶结点）




① p=top;

② top=top->next;

③ free(p);

## 3.1.3 栈的实现方式的比较

（1）顺序栈与链栈操作的时间复杂度均为O(1)。

（2）顺序栈主要的缺点是，需要预先定义栈空间的大小，当栈中的元素不多时容易造成空间的浪费，当栈的入栈操作过多时，又可能出现栈空间的不足。

（3）链栈的主要缺点是链栈需要增加每个结点的存储开销。

## 3.2 栈 的 应 用

3.2.1 平衡符号（自学）

3.2.2 表达式求值

3.2.3 函数调用（自学）

3.2.4 递归与栈（自学）

## 3.2.1 平衡符号

- 平衡符号—怎样匹配符号 ( )、 [ ]、{ }
- 左符号 “(”、 “[”、 “{”称为开分隔符(opening delimiter)，右符号 “)”、 “]”、 “}”称为闭分隔符(closing delimiter)。
- 平衡符号的算法主要分为以下3步：

(1) 设置空栈S。

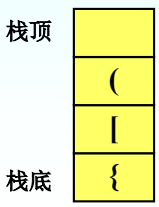
(2) 依次读入文件中的字符，如果读到的字符属于开分隔符，则将其压入到栈S中，如果读到的字符属于闭分隔符，则将栈S中的栈顶元素弹出，并与此闭分隔符比较，如果二者不匹配，则报错，否则继续读入下一个字符，直到文件结尾。

(3) 最后如果栈S为空栈，则说明符号匹配成功，否则匹配不成功，报错。

## 平衡符号的例题

例：判断表达式中的括号是否匹配？

{ 2 * [ 3 + （6 - 4） + 8 ] / 10 }




充分利用栈的“后进先出”特性，使得括号匹配

## 3.2.2 表达式的计算

### (1)表达式的求值

表达式通常是由*操作数*和*运算符*组成的。表达式的运算顺序取决于运算符的优先级，优先级大的运算符先进行处理。

运算符的按优先级从大到小排列：

*　　+

**　　#

/　　-

#表示整个表达式求值结束

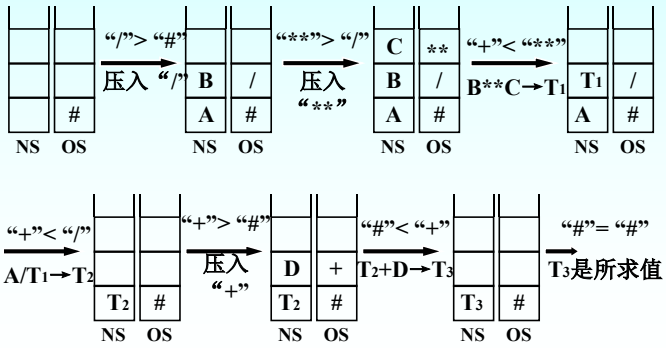
两个工作栈：
- NS栈——存放操作数及运算结果
- OS栈——存放运算符

## 表达式求值的基本思想

- 置NS栈为空栈，“#”作为OS栈的栈底元素
- 自左至右读入表达式中每个字符：
  - 若是操作数，则进NS栈
  - 若是运算符，则和OS栈的栈顶运算符比较优先级：
    - 若“>”，则压入OS栈
    - 若“≤ ”，则取NS栈两元素及运算符做运算，并存结果

## 表达式求值的例题

例：求表达式 A/B**C+D# 的值。




## (2)中缀表达式转换成后缀表达式

➢表达式的表示形式有中缀、前缀和后缀3种形式。

$$A \otimes B \Leftrightarrow AB \otimes$$

- 中缀表达式——即数学运算，按操作符的优先级进行计算。
- 后缀表达式——操作符在两个操作数之后。它的计算规则非常简单，严格按照从左到右的次序依次执行每一个操作。每遇到一个操作符，就将前面的两个数执行相应的操作。

## 中缀表达式和后缀表达式

| 中缀表达式 | 后缀表达式 |
|---|---|
| 3 / 5 + 8 | 3 5 / 8 + |
| 18 - 9 * ( 4 + 3 ) | 18 9 4 3 + * - |
| (25 + a)*(a*(a + b)+ b) | 25 a + a a b + * b + * |

## 中缀表达式转换成后缀表达式的过程

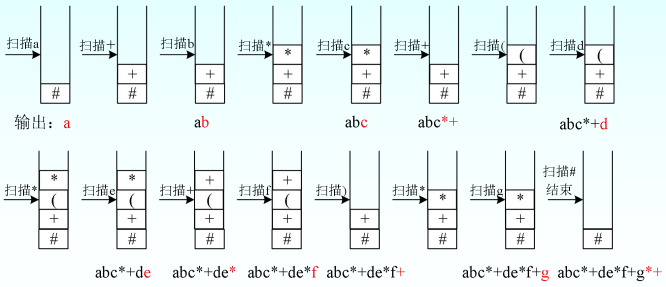
➢转换规则——操作数次序不变，操作符次序发生变化，同时去掉括号。

➢转换过程

- 设立一个栈，存放操作符，栈初始状态为#。
- 从左到右扫描中缀表达式：
  - 如果遇到操作数，直接输出；
  - 如果遇到操作符，与栈顶比较，当前操作符级别比栈顶操作符级别高则进栈，否则栈顶操作符出栈并输出；
  - 如果遇到左括号，进栈；如果遇到右括号，则一直出栈输出栈内操作符，直到出栈到左括号为止。

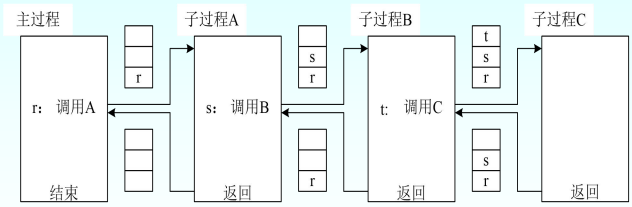
## 中缀表达式转换成后缀表达式的例题

例：分析将中缀表达式a+b*c+(d*e+f)*g#转换成后缀表达式的过程。




## 3.2.3 过程嵌套

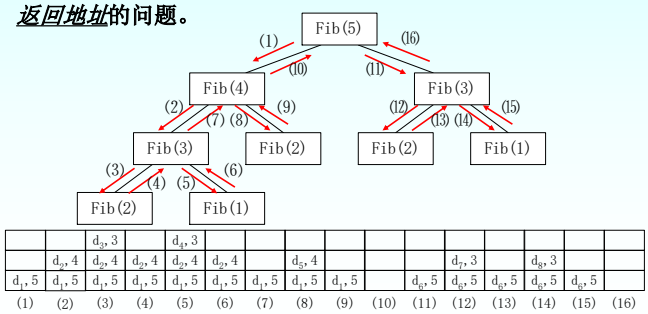
*嵌套*：将一个结构嵌入同种类型的另一个结构中。




## 3.2.4 过程递归

*过程递归*：一个过程通过一系列的过程调用语句，直接或间接调用自己的过程。

为了保证每次调用和返回能正确执行，必须解决*参数传递*和*返回地址*的问题。







## 过程递归的例题

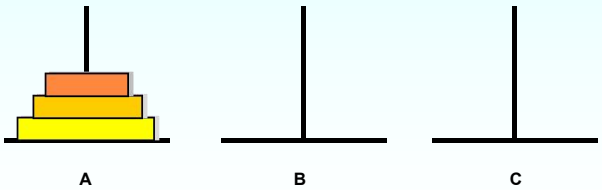
- 递归过程如下：

```
Fib（w）
{
 if（w！=0）
{
 Fib（w－1）；
 for（i=1；i<=w；i＋＋）
     输出w；
 输出换行；
}
}
```

若w=4，则调用该函数的结果是什么？

## 例：汉诺塔问题

将n个大小不同的圆盘从A借助于B移到C，遵守下列规则：（1）每次只能移动一个圆盘;（2）圆盘可以在3个塔座之间任意移动;（3）任何时刻，每个塔座上都不能将较大圆盘压在较小圆盘上。（以n=3为例）

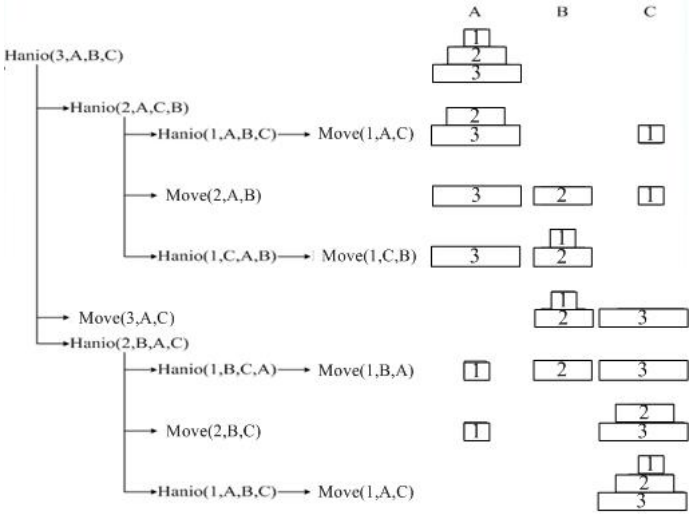



## 汉诺塔的算法

```
void main (void)
{
  int n;
  unsigned char a,b,c;
  n=3;
  a="A";  b="B";  c="C";
  Hanoi(n, a, b, c);
      :
      :
      :
}
```

```
void Hanoi(int n, char x, char y,
           char z)
{
   if(n==1)
     move(1,x,z);
   else
   {
     Hanoi(n-1,x,z,y);
     move(n,x,z);
     Hanoi(n-1,y,x,z);
   }
}
```

详细解答过程




## 3.3 队列

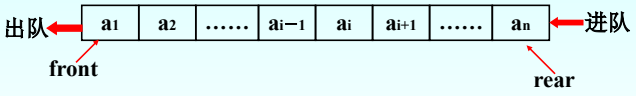

3.3.1 队列概述

3.3.2 队列的实现

3.3.3 队列实现方法比较


## 3.3.1 队列概述

队列——只能在一端进行插入、在另一端进行删除的线性表。




允许删除的一端为队首(front)，允许插入的一端为队尾(rear)。

特点：先进先出（FIFO）

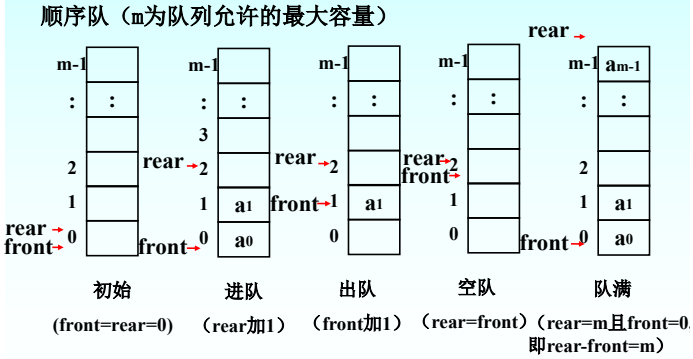
后进后出（LILO）

例：一个队列的入队序列是1、2、3、4，则队列的输出序列是_____。

A. 4 3 2 1　　B. 1 2 3 4　　C. 1 4 3 2　　D. 3 2 4 1

## 3.3.2 队列的实现——顺序队和链队

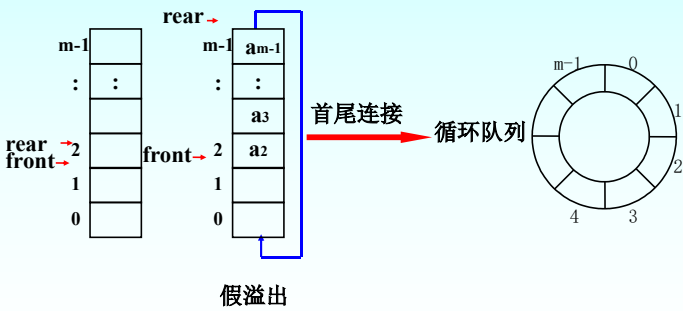
顺序队（m为队列允许的最大容量）



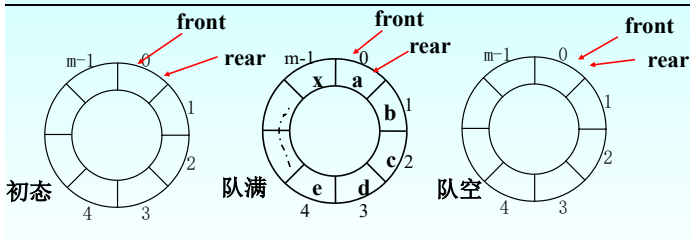

rear——指向队列尾元素的下一个位置；

front——指向队列头元素。

## 循环队列




## 循环队列的进队和出队



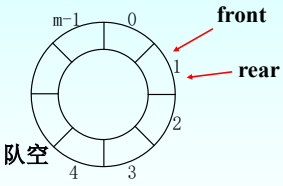

进队：(rear+1) % m　　出队：(front+1) % m

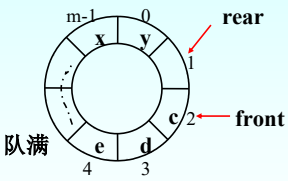
循环队列中队满/队空的判定条件：

rear＝front

出现二义！？

## 循环队列的队满与队空的判断




rear＝front

（rear+1）% m＝front

## 循环队列的例题

例：设循环队列的容量为40（序号从0到39），现经过一系列的入队和出队运算后，在下面两种情况中，循环队列中各有元素多少个？

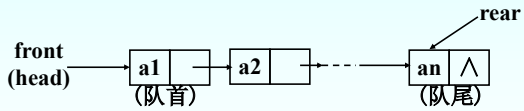
① front=11，rear=19

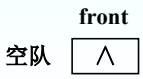
② front=19，rear=11

循环队列长度的计算公式：（rear-front+m）%m

## 4. 链队

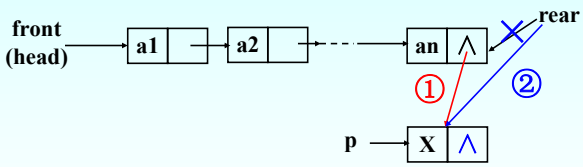
- 在链表表示的队中，
  链表的第一个结点是队首元素，由front指针指向；
  最后一个结点是队尾元素，由rear指针指向。
- 当front＝NULL时，表示队空。




## 链队的进队

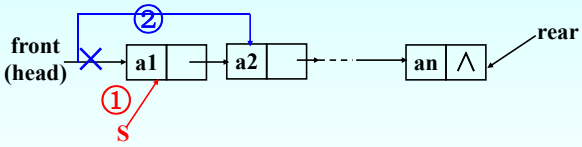
- 进队：新结点插入到链表尾部，作为最后一个结点。




```
p=(LinkedQNode) malloc (sizeof (LinkedQNode) );
p->data=x;
p->next=NULL;
rear->next=p;
rear=p;
```

## 链队的出队

- 出队：删除链表的第一个结点。




```
s=front;
front=front->next;
free(s);
```

## 运算受限的队列(自学)

*双向队列*：输入、输出可在两端进行。

a1,a2,…,an

*输出受限的队列*：输出运算仅在一端，输入运算可在两端进行。

a1,a2,…,an

*输入受限的队列*：输入运算仅在一端，输出运算可在两端进行。

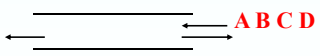
a1,a2,…,an

## 运算受限队列的例题

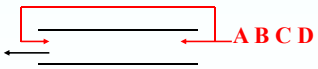
例：若输入序列为A、B、C、D，则
（1）能由输入受限队列得到、但不能由输出受限队列得到的输出序列是__DACB__。
（2）能由输出受限队列得到、但不能由输入受限队列得到的输出序列是__DBAC__。
（3）既不能由输入受限队列、又不能输出受限队列得到的输出序列是__DBCA__。

分析：

输入受限的队列：




输出受限的队列：




## 3.3.3 队列实现方法比较

(1)顺序队列的空间利用率不高，容易出现“假上溢”的现象。

(2)循环队列克服了顺序队列的“假上溢”现象，但是它和顺序队列一样，存储空间是事先分配好的。

(3)链队列克服了顺序队列空间固定的缺点，在对链队列进行入队操作时，不需要考虑上溢和队满问题。

(4)链队列和链表一样，需要增加结点的空间开销，并且还有运行malloc和free的开销。